



CareerLaunch

COMPLETE BUILD GUIDE

CareerLaunch

From business problem to live system —
every step, in order, no AI required.

A single, self-contained manual for rebuilding the entire CareerLaunch application in a fresh Salesforce Developer Edition org: the org itself, every object and field, every flow, the Apex and Lightning Web Components (referenced by name — source is supplied separately as the **Code Pack**), the payment gateway, certificates, error logging, multi-currency and multi-language, the Experience Cloud student portal, all permissions, the tests, and the release.

• Org & data model

• Automation & Apex

• Portal & payments

• Security · tests · release

CareerLaunch Platform

Enrolments · Invoicing · Certificates · Payments · Portal

Build Guide · Rev. August 2026

Oxfordable Careers — Salesforce Developer Programme

How this guide is organised

1 The business problem & the solution

The client brief · what we build · architecture · how to use the Code Pack

2 Workstation & org setup

Developer Edition org · CLI · VS Code · Git · project · org settings

3 The core data model

Course · Cohort · Enrolment · Contact field · formulas, roll-up, validation rules · sample data

4 Core automation — Flows

Confirmation actions · Quick Enrol wizard · scheduled reminder

5 Core Apex

Trigger + handler · queueable · invocable services · LWC controllers

6 Core UI — the Enrolment Console

Three Lightning Web Components · the App Page

7 Error logging framework

Error Log object · platform event · logger class · flow fault paths

8 Invoicing data model & configuration

Invoice object · validation · Exchange Rate & Integration Setting custom metadata

9 Multi-currency & multi-language

CurrencyService · ten custom labels · French translations

10 Payment gateway & automatic invoicing

Named Credential · interface, factory, stub, HTTP client · services · trigger wiring

11 Certificate PDF generation

Visualforce PDF page · controller · queueable service · completion flow

12 The Experience Cloud student portal

Portal controller & LWCs · site build · portal users · end-to-end checkpoint

13 Security & permissions reference

Both permission sets in full · the portal privacy model

14 Tests, UAT & release

Test suite · acceptance script · deployment · going live with payments

A Appendices

A — Code Pack manifest · B — master troubleshooting table · C — glossary

1 The business problem & the solution

Every build starts with a client, not a click. Read this chapter before touching Setup — everything you create later traces back to a sentence here.

1.1 The client brief — "Oxfordable CareerLaunch"

CLIENT: OXFORDABLE CAREERS

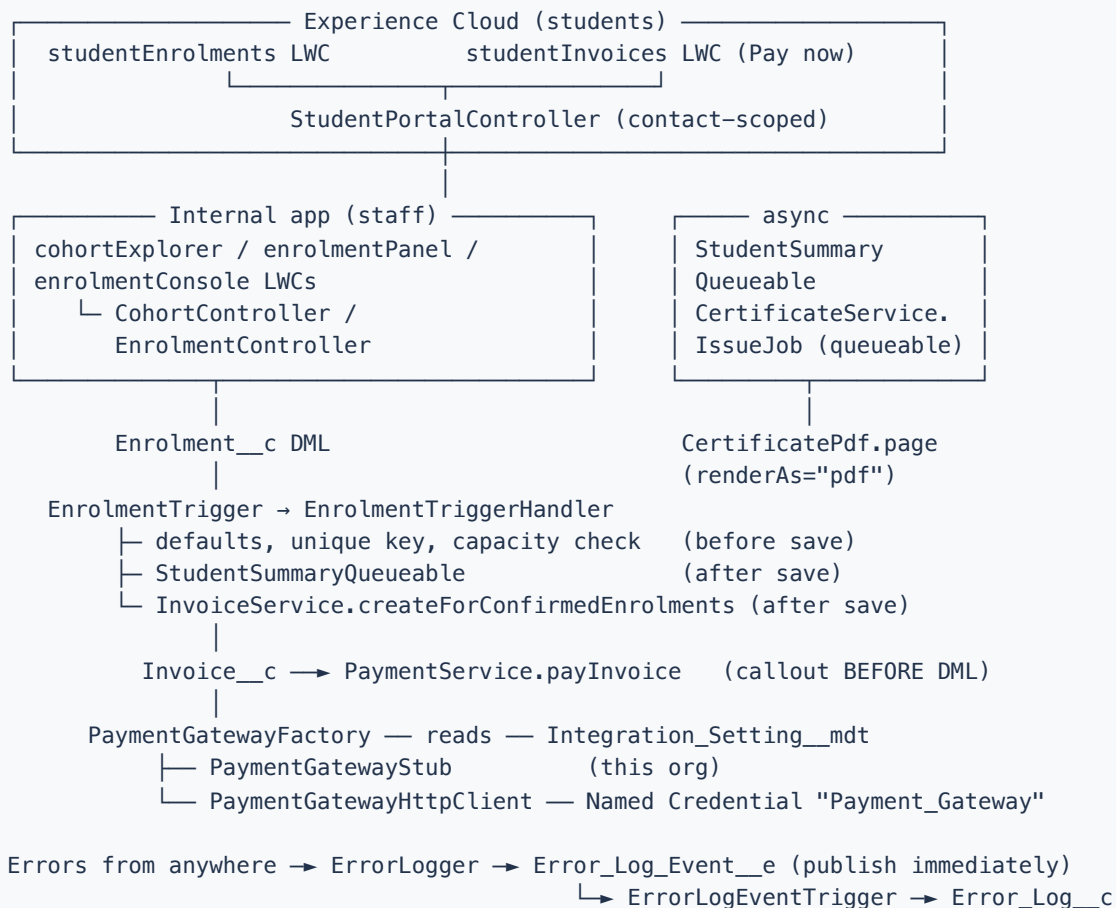
Oxfordable Careers runs a growing catalogue of technology training courses. Each course runs several times a year as a **cohort** with a fixed start date, a delivery mode and a limited number of seats. Today the team tracks everything in spreadsheets and WhatsApp: students get double-booked, cohorts get oversold, welcome emails are forgotten, invoices are typed by hand, certificates are made in a word processor, and nobody can answer "how many seats are left?" without opening three files.

They want **one Salesforce application** that manages courses, cohorts and enrolments; stops overselling; automates welcome and reminders; **raises and takes payment for invoices**; issues **completion certificates as PDFs**; gives students a **self-service portal** in their own language and currency; records **every error** somewhere staff can see; and gives the admissions team a one-screen console.

1.2 What we build — the full scope

LAYER	WHAT GETS BUILT	CHAPTER
Data model	Course, Cohort, Enrolment (+ Contact roll-up field), Invoice, Error Log, two custom metadata types	3 · 7 · 8
Automation	Welcome/confirmation flow, Quick Enrol wizard, scheduled reminder, auto-invoicing, certificate-on-completion, error fault paths	4 · 10 · 11
Apex	Trigger + handler (capacity, duplicates), async roll-up, invocable services, the payment layer, certificate job, error logger	5 · 7 · 10 · 11
UI	Enrolment Console (3 LWCs) for staff; 2 portal LWCs for students; a Visualforce PDF certificate	6 · 11 · 12
Integration	Payment gateway behind an interface — a deterministic stub in this org, a real HTTP client + Named Credential ready for production	10
i18n	Multi-currency via custom metadata rates; multi-language via custom labels + French translations	9
Delivery	Two permission sets, an Experience Cloud site, a test suite, a UAT script, CLI deployment	12 · 13 · 14

1.3 Architecture at a glance



THE LAYERING PRINCIPLE

The portal, the console and any future AI agent are only **channels**. They all call the same Apex service layer, over the same data model, protected by the same trigger. **Build the core once; expose it many times.**

1.4 How code works in this guide

You build every **object, field, tab, flow, label, setting and permission with clicks** — those steps are written out in full. For **Apex classes, triggers, the Visualforce page and Lightning Web Components**, this guide names the file and a Code Pack item, like this: **CODE PACK 10.4**. Ask your instructor for that item (or open the matching file in the project repository — Appendix A maps every item to its file), type or paste it exactly, and *read the comments* — they explain why the code is shaped the way it is.

BUILD IN CHAPTER ORDER

Later code calls earlier code (for example, the payment client calls the error logger). Follow the chapters in order and nothing will ever fail to compile. Do not skip checkpoints — they are how you catch a problem while it is still one step old.

2 Workstation & org setup

One-time setup: a free org that never expires, the professional toolchain, and a source-tracked project. Allow 45–60 minutes.

2.1 Create the org

- 1 Go to `developer.salesforce.com/signup` and sign up for a free **Developer Edition** org. Use an email address you have not used for Salesforce before (or add a plus-tag: `you+careerlaunch@gmail.com`).
- 2 Open the verification email, click the link, and set a password you will remember.
- 3 Write down your **username** — it looks like an email address but is not one.
- 4 Log in at `login.salesforce.com` and confirm you are in **Lightning Experience**.

2.2 Org settings (do these now, forget them later)

- 1 **Email deliverability:** Setup → Quick Find `Deliverability` → Access level = **All Email** → Save. (Without this, no flow email ever arrives.)
- 2 **My Domain:** Setup → Quick Find `My Domain` — Developer orgs come with one; just note the domain name, you will see it in URLs.

2.3 Install the toolchain

TOOL	WHERE	VERIFY WITH
Google Chrome	<code>google.com/chrome</code>	—
Visual Studio Code	<code>code.visualstudio.com</code>	opens
Salesforce Extension Pack	VS Code → Extensions → search "Salesforce Extension Pack" → Install	extension listed
Salesforce CLI	<code>developer.salesforce.com/tools/salesforcecli</code>	<code>sf --version</code>
Git + GitHub account	<code>git-scm.com</code> · <code>github.com</code>	<code>git --version</code>

2.4 Create the project and connect the org

```
sf --version
sf project generate --name careerlaunch
cd careerlaunch
sf org login web --alias devorg --set-default    # browser opens – log in
sf org list                                     # devorg should be listed and default
```

Open the `careerlaunch` folder in VS Code (**File** → **Open Folder...**). From now on: clicks happen in the browser, code happens here, and the CLI moves things between the two.

```
git init && git add . && git commit -m "Empty CareerLaunch project"
```

CHECKPOINT

`sf org open` opens your org in the browser without asking for a password. If it does, everything is connected.

3 The core data model

Three custom objects plus one field on Contact. Field API names must match this chapter **exactly** — every line of Apex depends on them. British spelling throughout: `Enrolment__c`, one l.

3.1 The relationships — and why

Course__c 1 —< N Cohort__c 1 —< N Enrolment__c N >— 1 Contact
(Lookup) (Master-Detail) (Lookup)

RELATIONSHIP	TYPE	WHY
Cohort → Course	Lookup	A cohort should survive independently; no cascade delete, no roll-up needed here.
Enrolment → Cohort	Master-Detail	We want a roll-up of enrolments on the cohort; an enrolment is meaningless without its cohort; cascade delete is desirable; sharing is controlled by the parent.
Enrolment → Contact	Lookup	A contact may exist with no enrolments and must never be deleted when one is.

3.2 Create the Course object

Setup → Object Manager → Create ▾ → Custom Object.

SETTING	VALUE
Label / Plural	Course / Courses
Record Name / Type	Course Name · Text
Tick	Allow Reports · Allow Activities · Track Field History · Launch New Custom Tab Wizard

Save → in the tab wizard pick any style → Next → Next → Save. Then **Fields & Relationships** → **New** for each row (each field walks four screens: type → details → field-level security (defaults) → layouts (ticked); use *Save & New*):

FIELD LABEL	TYPE	SETTINGS	API NAME CREATED
Course Level	Picklist	Values one per line: <code>Introductory</code> , <code>Advanced</code> . Restricted.	<code>Course_Level__c</code>
Duration (Weeks)	Number	Length 2, decimals 0	<code>Duration_Weeks__c</code>
Fee	Currency	Length 10, decimals 2	<code>Fee__c</code>
Active	Checkbox	Default: Checked	<code>Active__c</code>
Description	Text Area (Long)	Length 1000	<code>Description__c</code>

3.3 Create the Cohort object

SETTING	VALUE
---------	-------

Label / Plural	Cohort / Cohorts
Record Name / Type	Cohort Number · Auto Number · format C0-{0000} · start 1
Tick	Allow Reports · Allow Activities · Launch New Custom Tab Wizard

Fields — leave the three calculated ones for \$3.5 (they cannot be built until the master-detail exists):

FIELD LABEL	TYPE	SETTINGS	API NAME
Course	Lookup Relationship	Related To: Course · tick Required	Course__c
Start Date	Date	tick Required	Start_Date__c
End Date	Date	—	End_Date__c
Delivery Mode	Picklist	Online , In-Person , Hybrid	Delivery_Mode__c
Capacity	Number	Length 3, decimals 0, default 20 , Required	Capacity__c
Status	Picklist	Planned , Open , Full , Running , Completed , Cancelled	Status__c

3.4 Create the Enrolment object

SETTING	VALUE
Label / Plural	Enrolment / Enrolments
Record Name / Type	Enrolment Number · Auto Number · format ENR-{00000}
Tick	Allow Reports · Allow Activities · Launch New Custom Tab Wizard

Fields — **in this order** (the Master-Detail must exist before the roll-up can be built):

FIELD LABEL	TYPE	SETTINGS	API NAME
Cohort	Master-Detail	Related To: Cohort · accept sharing defaults	Cohort__c
Student	Lookup Relationship	Related To: Contact · tick Required	Student__c
Status	Picklist	Applied (default), Confirmed , Waitlisted , Cancelled , Completed . Restricted.	Status__c
Enrolment Date	Date	—	Enrolment_Date__c
Amount Paid	Currency	Length 10, decimals 2, default 0	Amount_Paid__c
Enrolment Key	Text	Length 64 · tick External ID and Unique (case-insensitive)	Enrolment_Key__c

Attendance %	Percent	Length 3, decimals 0	Attendance_Percent__c
Certificate Issued	Checkbox	Default: Unchecked	Certificate_Issued__c

PAUSE — WHAT THE MASTER-DETAIL JUST CHANGED

The Cohort now **owns** the Enrolment: deleting a Cohort cascade-deletes its Enrolments, roll-up summaries become possible on Cohort, and **who can see an Enrolment is controlled by the parent**. That last fact matters again in Chapter 12.

3.5 Back on Cohort — the three calculated fields

FIELD LABEL	TYPE	SETTINGS	API NAME
Enrolled Count	Roll-Up Summary	Summarised object Enrolments · COUNT · filter: Status <i>not equal to</i> Cancelled	Enrolled_Count__c
Seats Remaining	Formula → Number, 0 dp	Capacity__c - Enrolled_Count__c	Seats_Remaining__c
Cohort Label	Formula → Text	Course__r.Name & " - " & TEXT(Start_Date__c)	Cohort_Label__c

3.6 The Contact field & the two validation rules

- Object Manager** → **Contact** → **Fields & Relationships** → **New** → Number, length 3, decimals 0, label **Total Enrolments** → creates **Total_Enrolments__c**.
- Object Manager** → **Cohort** → **Validation Rules** → **New** — name **End_Date_After_Start_Date**, formula `AND(NOT(ISBLANK(End_Date__c)), End_Date__c < Start_Date__c)`, message *End Date cannot be earlier than Start Date*.
- Object Manager** → **Enrolment** → **Validation Rules** → **New** — name **Payment_Not_Above_Fee**, formula `Amount_Paid__c > Cohort__r.Course__r.Fee__c`, message *Amount Paid cannot exceed the course fee*. (A cross-object formula spanning two relationships.)
- Page layouts**: on Cohort drag *Enrolled Count*, *Seats Remaining*, *Cohort Label* onto the layout and confirm the *Enrolments* related list is present. On Enrolment, confirm the *Invoices* related list once Chapter 8 exists.

3.7 Sample data — do not skip

OBJECT	CREATE
Course	Salesforce Developer – Introductory (Introductory, 3 wks, fee 500, Active) and Salesforce Developer – Advanced (Advanced, 10 wks, fee 1500, Active)
Cohort	CO-0001 — Introductory course · starts today + 14 · Capacity 3 · Open CO-0002 — Advanced course · starts today + 30 · Capacity 10 · Open
Contact	Three students with real-looking email addresses (automation searches on them)

CHECKPOINT

Create one Enrolment linking a Contact to CO-0001. Open CO-0001: **Enrolled Count = 1** and **Seats Remaining = 2** — with no automation at all. Then try a Cohort whose End Date is before its Start Date: the validation rule must block it.

4 Core automation — Flows

Declarative first: use clicks where clicks are enough, code where code is necessary. These three flows are the "clicks are enough" half; Chapter 5 is the other half.

4.1 Record-triggered flow — `Enrolment_Confirmed_Actions`

Setup → Flows → New Flow → Record-Triggered Flow.

SETTING	VALUE
Object / Trigger	<code>Enrolment__c</code> · created or updated
Entry conditions	<code>Status__c</code> Equals <code>Confirmed</code> · run <i>only when updated to meet the conditions</i>
Optimise for	Actions and Related Records (after-save — we touch other records)

- Get Records** `Get_Cohort` — `Cohort__c` where `Id = {!$Record.Cohort__c}`; first record only; fields `Seats_Remaining__c`, `Status__c`, `Start_Date__c`.
- Action → Send Email** — to `{!$Record.Student__r.Email}`; subject `Your place is confirmed — {!$Record.Cohort__r.Cohort_Label__c}`; body: welcome text + start date.
- Create Records** — a Task: subject `Send welcome pack to {!$Record.Student__r.Name}`, WhatId `{!$Record.Id}`, OwnerId `{!$Record.OwnerId}`, due `{!$Flow.CurrentDate}`, priority High.
- Decision** `Is_Cohort_Now_Full` — Yes when `{!Get_Cohort.Seats_Remaining__c} ≤ 0`.
- Update Records** on Yes — update the record from `Get_Cohort`, set `Status__c = Full`.
- Save as** `Enrolment Confirmed Actions` → **Activate**. A saved flow that is not activated does nothing.

4.2 Screen flow — `Quick_Enrol_Student`

ELEMENT	CONTENT
Screen 1	Picklist choice set from active <code>Course__c</code> records — "Which course?"
Get Records	<code>Cohort__c</code> where <code>Course = selection</code> AND <code>Status__c = Open</code> , sorted by Start Date, all records
Screen 2	Radio buttons from the cohort collection — show <code>Cohort_Label__c</code> , seats remaining in help text
Screen 3	Lookup component for Contact — "Which student?"
Action	Apex action Find Available Cohorts (built in Chapter 5) — re-check seats before writing
Decision + Create	Seats available? → Create <code>Enrolment__c</code> (Cohort, Student, Status = Confirmed)
Screen 4 (Yes / No)	Confirmation with the enrolment number / "that cohort just filled up" with Back
Fault paths	Every Get/Create gets a Fault connector to an error screen showing <code>{!\$Flow.FaultMessage}</code> — Chapter 7 adds error logging here too

Save as `Quick Enrol Student`, **Activate**, and surface it with a button or from the App Launcher. (If you have not built Chapter 5 yet, leave the Apex Action out and add it afterwards.)

4.3 Scheduled flow — Cohort_Start_Reminder

SETTING	VALUE
Trigger	Schedule-Triggered · daily 07:00
Object + conditions	Enrolment__c where Status__c = Confirmed AND Cohort__r.Start_Date__c = today + 3 (formula resource)
Actions	Send a reminder email and create a Task

CHECKPOINT

Set an existing enrolment's Status to **Confirmed**: the student's email arrives (deliverability = All Email!), a Task appears on the record — and when the last seat of CO-0001 fills, the Cohort flips itself to **Full**.

5 Core Apex

The code half of the automation matrix: things clicks genuinely cannot do — counting sibling records in bulk, database-enforced duplicate prevention, asynchronous aggregation, and services that Flow (and later a portal) can call.

5.1 Which tool, and why — the decision matrix

REQUIREMENT	TOOL CHOSEN	WHY NOT THE ALTERNATIVE
Seat count on a cohort	Roll-up + formula	No automation needed at all — always ask "can a field do this?" first
Welcome email + task	Record-triggered flow	Declarative, no test class, admin-editable. Apex would be over-engineering
Prevent overbooking	Apex trigger (before)	A validation rule cannot count sibling records; Flow cannot cleanly block a 200-record load per record
Prevent duplicates	Apex + unique External ID	Database-enforced, bulk-safe, free at runtime
Contact roll-up count	Queueable Apex	Aggregate after the transaction commits, with fresh governor limits
Availability / status Q&A	Invocable Apex	One method, many consumers: screen flow, agent, anything

5.2 How to create an Apex class or trigger

VS Code: Command Palette (`Cmd/Ctrl-Shift-P`) → `SFDX: Create Apex Class` (or *Apex Trigger*) → name it → file opens → replace the skeleton with the Code Pack item → save → right-click the file → **SFDX: Deploy Source to Org**. **Developer Console** (fallback): File → New → Apex Class / Apex Trigger.

5.3 Build these seven, in order

#	CREATE	CODE PACK	WHAT IT DOES — READ FOR THESE THINGS
1	Trigger <code>EnrolmentTrigger</code> on <code>Enrolment__c</code> (before/after, insert/update)	5.1	Routing only — no logic in triggers ; every branch calls the handler
2	Class <code>EnrolmentTriggerHandler</code>	5.2	<code>setDefaultts</code> (date/status/amount) · <code>buildUniqueKey</code> = CohortId-StudentId into the unique External ID (duplicates die at the database) · <code>preventOverbooking</code> — the bulk shape to memorise: collect ids → ONE query → allocate in memory → addError() · <code>enqueueStudentSummary</code>
3	Class <code>StudentSummaryQueueable</code>	5.3	Aggregate query → update <code>Contact.Total_Enrolments__c</code> ; asynchronous = fresh governor limits

4	Class <code>CohortAvailabilityService</code>	5.4	<code>@InvocableMethod</code> "Find Available Cohorts" — the bridge Flow (and any agent) calls
5	Class <code>EnrolmentStatusService</code>	5.5	<code>@InvocableMethod</code> "Check Enrolment Status" by student email
6	Class <code>CohortController</code>	5.6	<code>@AuraEnabled(cacheable=true)</code> reads for the LWCs — cacheable is what makes <code>@wire</code> possible
7	Class <code>EnrolmentController</code>	5.7	Plain <code>@AuraEnabled</code> write; <code>DmlException</code> → readable sentence via <code>AuraHandledException</code> , never a stack trace

CHECKPOINT — TWO ACCEPTANCE CRITERIA, LIVE

Enrol a 4th student onto CO-0001 (capacity 3): the save must fail with *your* capacity message. Enrol the same student onto the same cohort twice: blocked by the duplicate key. Then finish §4.2 — add the **Find Available Cohorts** Apex action to the Quick Enrol flow.

6 Core UI — the Enrolment Console

Three Lightning Web Components on one App Page. An LWC is three files — template (.html), class (.js), metadata (.js-meta.xml) — and it can only be created from VS Code, never from Setup.

6.1 Create the components

VS Code → Command Palette → SFDX: Create Lightning Web Component → folder force-app/main/default/lwc . Create each, replace all three generated files with the Code Pack item, then deploy the lwc folder.

COMPONENT	CODE PACK	TEACHING BEAT
cohortExplorer	6.1	@wire for reactive reads; datatable; row selection dispatches a cohortselected CustomEvent — data down, events up
enrolmentPanel	6.2	@api public property receives the cohort; imperative Apex for the write; ShowToastEvent ; NavigationMixin
enrolmentConsole	6.3	Thin wrapper wiring the two siblings; calls explorer.refresh() (which calls refreshApex) after an enrolment

6.2 The App Page

- 1 Setup → Lightning App Builder → New → App Page → name Enrolment Console → two-column layout.
- 2 Drag enrolmentConsole onto the page → Save → Activate → add to navigation (e.g. the Sales app), so it appears in the App Launcher.

CHECKPOINT — THE WHOLE ARCHITECTURE IN ONE ACTION

Enrol a student from the console: a toast appears, the seat count drops, and an attempted overbook is still blocked — the Chapter 5 trigger, surfaced through the new UI. The console is just another channel on the same protected core.

7 Error logging framework

One front door for every error — flows, triggers, Apex, integrations — that survives transaction rollback. Built now because Chapters 10 and 11 call it.

7.1 The idea

WHY AN EVENT, NOT AN INSERT

If a failing transaction rolls back, a plain `insert Error_Log__c` rolls back with it — the error erases its own evidence. So the code publishes a **publish-immediately platform event** (which escapes the rollback); a trigger on the event then writes the permanent record in a fresh transaction, as the Automated Process user.

```
code / flow fault → ErrorLogger → Error_Log_Event__e (publish immediately)
                                ↳ ErrorLogEventTrigger → Error_Log__c (+ tab)
```

7.2 Create the Error Log object

Object Manager → **Create** ▾ → **Custom Object** — Label `Error Log` / `Error Logs`, Record Name `Error Number`, Auto Number `ERR-{00000}`, tick Allow Reports + **tab wizard**. Then the fields:

FIELD LABEL	TYPE	SETTINGS	API NAME
Source Type	Picklist	Apex (default), Trigger, Flow, Integration — restricted	Source_Type__c
Source Name	Text	255	Source_Name__c
Error Message	Text Area (Long)	32768 · 5 lines	Error_Message__c
Stack Trace	Text Area (Long)	32768 · 10 lines	Stack_Trace__c
Record Id	Text	18	Record_Id__c
Severity	Picklist	Info, Warning, Error (default), Critical — restricted	Severity__c

7.3 Create the platform event

Setup → **Quick Find** `Platform Events` → **New Platform Event** — Label `Error Log Event` / `Error Log Events`, Object Name `Error_Log_Event`, Publish Behavior = **Publish Immediately**. Then its fields (platform events offer fewer types — that is normal):

FIELD LABEL	TYPE	SETTINGS	API NAME
Source Type	Text	50	Source_Type__c
Source Name	Text	255	Source_Name__c
Error Message	Text Area (Long)	32768	Error_Message__c

Stack Trace	Text Area (Long)	32768	Stack_Trace__c
Record Id	Text	18	Record_Id__c
Severity	Text	20	Severity__c

7.4 The logger and its trigger

- 1 Create class `ErrorLogger` — **7.1**. Find the three entry points: Apex overloads `log(e, 'ClassName'[, recordId])`; the `@InvocableMethod` **Log Flow Error**; and the single private publisher — every path ends in `EventBus.publish`, never `insert`.
- 2 Create trigger `ErrorLogEventTrigger` on `Error_Log_Event__e` (after insert) — **7.2**. It copies each event into an `Error_Log__c` row.
- 3 **Wire the flow fault paths:** in `Quick_Enrol_Student` (and any flow with fault connectors), drag an Action → **Log Flow Error** onto each fault path before the error screen: Flow Name = the flow's name as text, Error Message = `{!$Flow.FaultMessage}`. Save as new version → Activate.

CHECKPOINT — PROVE THE ROLLBACK SURVIVAL

Developer Console → Debug → Open Execute Anonymous Window:

```
try { Integer boom = 1 / 0; }
catch (Exception e) { ErrorLogger.log(e, 'CheckpointTest'); }
```

Open the **Error Logs** tab: ERR-00001 exists with type Apex, source CheckpointTest, message "Divide by 0" and a stack trace.

8 Invoicing data model & configuration

The Invoice object that Chapter 10 automates, plus two Custom Metadata Types that turn currency rates and the gateway choice into admin-editable configuration.

8.1 Create the Invoice object

Object Manager → **Create** ▾ → **Custom Object** — Label `Invoice` / `Invoices`, Record Name `Invoice Number`, Auto Number `INV-{{00000}}`, tick Allow Reports · Allow Activities · **tab wizard**. Fields, **Master-Detail** first:

FIELD LABEL	TYPE	SETTINGS	API NAME
Enrolment	Master-Detail	Related To Enrolment · related list label <code>Invoices</code> · accept sharing defaults	<code>Enrolment__c</code>
Amount (GBP)	Currency	12,2 · tick Required	<code>Amount__c</code>
Currency Code	Picklist	<code>GBP</code> (default), <code>USD</code> , <code>EUR</code> — restricted	<code>Currency_Code__c</code>
Exchange Rate	Number	12,6 · default <code>1</code> · description: <i>rate frozen at issue time — 1 GBP = this many units</i>	<code>Exchange_Rate__c</code>
Amount (Invoice Currency)	Formula → Number, 2 dp	<code>Amount__c * Exchange_Rate__c</code> · blanks as zeroes	<code>Amount_In_Currency__c</code>
Status	Picklist	<code>Draft</code> (default), <code>Sent</code> , <code>Paid</code> , <code>Cancelled</code> — restricted	<code>Status__c</code>
Due Date / Paid Date	Date · Date	—	<code>Due_Date__c</code> / <code>Paid_Date__c</code>
Payment Reference	Text	100 · description: <i>gateway transaction id</i>	<code>Payment_Reference__c</code>

WHY TWO AMOUNTS AND A FROZEN RATE

`Amount__c` is always GBP, the org's one true currency. The rate is **stamped onto the invoice at issue time** and the display amount is a formula over the two — so an admin editing tomorrow's rates can never change what an already-issued invoice is worth.

Validation rule — name `Paid_Requires_Reference`, formula `AND(ISPICKVAL(Status__c, "Paid"), ISBLANK(Payment_Reference__c))`, message *An invoice cannot be marked Paid without the gateway payment reference.*

8.2 Custom Metadata Type — Exchange Rate

- Setup** → **Quick Find** `Custom Metadata Types` → **New Custom Metadata Type** — Label `Exchange Rate` / `Exchange Rates`, Object Name `Exchange_Rate` → Save.
- Custom Fields** → **New**: `Rate From GBP` — Number 12,6, Required → `Rate_From_GBP__c`; and `Symbol` — Text 5 → `Symbol__c`.
- Manage Exchange Rates** → **New**, three records: **GBP** = 1.0 / £ · **USD** = 1.27 / \$ · **EUR** = 1.17 / € (label and name identical, e.g. `USD`).

8.3 Custom Metadata Type — Integration Setting

- 1 Same route — Label `Integration Setting / Integration Settings` , Object Name `Integration_Setting` .
- 2 One field: `Implementation Class` — Text 255, Required → `Implementation_Class__c` .
- 3 One record — Label `Payment Gateway` , Name `Payment_Gateway` , Implementation Class = `PaymentGatewayStub` . (That class does not exist until Chapter 10 — the record is just text until the factory reads it. This record is the **switch** that one day flips the org from stub to a real provider with zero code changes.)

8.4 Give staff access

Open **Setup** → **Permission Sets** → **Enrolment Console User** (created in Chapter 13 if you have not made it yet — you can return here): Object Settings → **Invoices** → Read/Create/Edit (no Delete), Read on all fields (Edit on the editable ones), tab Available + Visible.

CHECKPOINT

Invoices tab → New → pick an Enrolment, Amount 500, Save: Currency defaults GBP, rate 1.000000, **Amount (Invoice Currency) = 500.00**. Edit → Status = Paid → Save: **blocked** — no payment reference. Delete this manual test invoice.

9 Multi-currency & multi-language

A service class over the rate records, and every piece of student-facing text moved into Custom Labels so the app can speak French without touching code.

9.1 CurrencyService

Create class `CurrencyService` — **9.1**. Note: custom metadata is read with `Exchange_Rate__mdt.getInstance('USD')` — **no SOQL**; money is rounded 2 dp half-up; unknown currencies throw a named exception. Quick test in Execute Anonymous:

```
System.debug(CurrencyService.convertFromGBP(100, 'USD')); // 127.00
System.debug(CurrencyService.format(1250, 'GBP')); // £1,250.00
```

9.2 The ten custom labels

Setup → **Quick Find** `Custom Labels` → **New Custom Label** — create each row exactly (Chapters 11–12 import them **by name**; a typo here is a compile error there). Category `CareerLaunch` on all.

NAME (EXACT)	VALUE	NAME (EXACT)	VALUE
<code>Certificate_Title</code>	Certificate of Completion	<code>Portal_My_Enrolments</code>	My Enrolments
<code>Certificate_Awarded_To</code>	This certificate is proudly awarded to	<code>Portal_My_Invoices</code>	My Invoices
<code>Certificate_Completion_Text</code>	for successfully completing	<code>Portal_Pay_Now</code>	Pay now
<code>Certificate_Issued_On</code>	Issued on	<code>Portal_No_Records</code>	Nothing to show here yet.
<code>Portal_Download_Certificate</code>	Download certificate	<code>Portal_Payment_Success</code>	Payment received - thank you!

9.3 Enable French and translate

- 1 Setup** → **Quick Find** `Translation Language Settings` → Enable Translation Workbench → **Add** → French → tick Active → add yourself as translator → Save.
- 2 Setup** → **Quick Find** `Translate` → Language French · Component **Custom Labels** → double-click the Translation column per row:

LABEL	FRENCH	LABEL	FRENCH
<code>Certificate_Title</code>	Certificat de réussite	<code>Portal_My_Enrolments</code>	Mes inscriptions
<code>Certificate_Awarded_To</code>	Ce certificat est fièrement décerné à	<code>Portal_My_Invoices</code>	Mes factures
<code>Certificate_Completion_Text</code>	pour avoir terminé avec succès	<code>Portal_Pay_Now</code>	Payer maintenant

Certificate_Issued_On

Délivré le

Portal_No_Records

Rien à afficher pour le moment.

Portal_Download_Certificate

Télécharger le certificat

Portal_Payment_Success

Paiement reçu - merci !

HOW THIS PAYS OFF

A label used as `{!$Label.Certificate_Title}` in Visualforce, or imported into an LWC via `@salesforce/label/c. ...` , resolves **in the viewing user's language** automatically. One codebase, every language you translate.

10 Payment gateway & automatic invoicing

The full payment stack: an interface, a metadata-driven factory, a deterministic stub for this org, a production-ready HTTP client, and the services that raise and settle invoices. The most interview-valuable chapter in the guide.

THE SEAM PRINCIPLE — SAY IT OUT LOUD

Everything above the interface knows nothing about HTTP; everything below it knows nothing about enrolments. Swapping the stub for a real provider is a **custom-metadata edit**, not a code change. This is how real teams stub an external system they cannot reach from a training org.

10.1 The Named Credential

- 1 **Setup** → **Quick Find** `Named Credentials` → dropdown next to New → **New Legacy**.
- 2 Label `Payment Gateway` · Name `Payment_Gateway` · URL `https://api.payments.example.com` · Identity Type **Anonymous** · Protocol **No Authentication** → Save.

The credential owns the URL and (in production) the secrets, so Apex never contains either. The URL is a placeholder — nothing calls it while the stub is configured.

10.2 Build the eight pieces, in order

#	CREATE	CODE PACK	READ THE CODE FOR...
1	Class <code>PaymentRequest</code>	10.1	A plain data bag — amount, currency, invoice number, email, idempotency key . No SObjects: the gateway layer must not know the data model
2	Class <code>PaymentResult</code>	10.2	<code>success / transactionId / errorMessage</code> + factory methods <code>ok()</code> and <code>fail()</code>
3	Class <code>IPaymentGateway</code>	10.3	The contract: one method, <code>charge(request)</code> , zero implementation
4	Class <code>PaymentGatewayStub</code>	10.4	Deterministic: rejects amount ≤ 0 ; an email containing <code>declined</code> simulates a failed card; else succeeds with <code>STUB-<invoice>-<ccy></code>
5	Class <code>PaymentGatewayFactory</code>	10.5	Reads the §8.3 record → <code>Type.forName().newInstance()</code> → verifies the interface → clear error on admin typo. <code>@TestVisible</code> override for tests
6	Class <code>PaymentGatewayHttpClient</code>	10.6	Endpoint <code>callout:Payment_Gateway/v1/charges</code> — never a raw URL; the Idempotency-Key header (a retry after a timeout can never double-charge); <code>CalloutException</code> → <code>ErrorLogger</code> Critical; wire format hidden in private inner classes
7	Class <code>InvoiceService</code>	10.7	For every Confirmed enrolment: one <code>Sent</code> invoice, course fee, due +14 days — skipping free courses and enrolments that already hold a live invoice, so it is idempotent . Bulk shape: ids → one query → build → one insert

Load → refuse Paid/Cancelled → build request → factory → `charge()` → on success mark Paid + roll into `Amount_Paid__c`.
Notice: **callout before DML** (platform rule), and `without sharing` with a comment explaining why (system action; amount always from the invoice, never the caller)

10.3 One line in the trigger handler

Open `EnrolmentTriggerHandler` and inside `onAfterChange`, directly after

`enqueueStudentSummary(newEnrolments);`, add the call to

`InvoiceService.createForConfirmedEnrolments(newEnrolments);` — 10.9 shows the exact two lines (call + comment). Deploy.

CHECKPOINT 1 — THE INVOICE RAISES ITSELF

Create an Enrolment with Status **Confirmed** → its Invoices related list shows one invoice: Sent, fee amount, due +14, rate 1. Re-save the enrolment with any edit → **still exactly one invoice**.

CHECKPOINT 2 — PAY IT, DECLINE IT, DOUBLE-PAY IT

Execute Anonymous (use your invoice's id): `PaymentResult r = PaymentService.payInvoice('a0X...');`
`System.debug(r.success + ' / ' + r.transactionId);`

Expected `true / STUB-INV-0000x-GBP`; the invoice flips to Paid with reference and date; the enrolment's Amount Paid equals the fee. Then enrol a contact whose email contains `declined` and pay their invoice: `false`, invoice stays Sent. Pay the first invoice again: a clear *already paid* exception.

11 Certificate PDF generation

A designed A4 certificate rendered as a real PDF, stored as a File on the enrolment, triggered by a flow when the student completes.

THE PATTERN IN ONE PARAGRAPH

A Visualforce page with `renderAs="pdf"` is the platform's built-in PDF engine. Turning the page into a **stored file** uses `getContent()`, which counts as a callout and is banned in triggers — so the render happens **asynchronously**, in a Queueable that declares `Database.AllowsCallouts`. Failures are logged via Chapter 7 (async work has no user watching).

11.1 Build the three pieces

- 1 Create class `CertificateController` — 11.1. It reads the `?id=` page parameter and queries the enrolment with student, cohort and course names. Thin on purpose.
- 2 Create the Visualforce page: **Setup** → **Quick Find** **Visualforce Pages** → **New** — Label `Certificate PDF`, Name **exactly** `CertificatePdf` (the Queueable references `Page.CertificatePdf`). Replace the markup with 11.2. Find in it: `renderAs="pdf"`, the A4-landscape `@page` CSS, and the four `{!$Label.Certificate_...}` merge fields — a French student receives a French certificate.
- 3 **See it now:** browse to `https://<your-domain>/apex/CertificatePdf?id=<enrolment id>` (copy the 18-character id from any Enrolment URL) — a PDF renders in the browser.
- 4 Create class `CertificateService` — 11.3. Find: the `@InvocableMethod` **Issue Completion Certificate**; the inner `IssueJob` queueable; the `Certificate_Issued__c = false` filter (never issued twice); `ContentVersion` with `FirstPublishLocationId` (files *and* links the PDF in one insert); try/catch → `ErrorLogger`; and the `Test.isRunningTest()` guard (`getContent()` is blocked in unit tests).

11.2 The completion flow

- 1 **Setup** → **Flows** → **New Flow** → **Record-Triggered Flow** — Object `Enrolment__c`; created or updated; entry `Status__c = Completed`; only when updated to meet conditions; after-save.
- 2 One element: **Action** → **Issue Completion Certificate**, Enrolment Id = `{!$Record.Id}`.
- 3 Fault connector from the action → **Log Flow Error** (flow name + `{!$Flow.FaultMessage}`).
- 4 Save as `Issue_Certificate_On_Completion` → **Activate**.

CHECKPOINT

Set a Confirmed enrolment to **Completed** → wait a few seconds → refresh: *Certificate Issued* ticked · Files holds `Certificate ENR-0000x` · the PDF shows the student's name, the course and today's date. Flip the status away and back: **no second file**. If nothing appears, the answer is in the Error Logs tab — that is what it is for.

12 The Experience Cloud student portal

A public-facing site where each student sees only their own records, pays with one click, and downloads their certificate. Read the security model first — it is the design decision of the chapter.

THE SECURITY MODEL — THREE RULES, ALL SERVER-SIDE

1. Every query starts from the **logged-in user's Contact** — the browser never says whose data to load. 2. `payInvoice` re-checks ownership on the server before money moves. 3. The controller is `without sharing`, deliberately: `Enrolment__c` is a master-detail child (sharing controlled by its parent Cohort, which has no contact field), so Experience Cloud *sharing sets cannot reach it*. Row-level privacy lives in the contact-scoped WHERE clauses instead — and there is a test that proves it in Chapter 14.

12.1 Controller and components

- 1 Create class `StudentPortalController` — 12.1. Find: `resolveContactId()` (User → ContactId); two `@AuraEnabled(cacheable=true)` reads returning small view-wrapper classes; the ownership `COUNT()` gate inside `payInvoice`; the `@TestVisible contactIdOverride` for tests.
- 2 VS Code → SFDX: Create Lightning Web Component → `studentEnrolments`; replace all three files with 12.2.
- 3 Repeat for `studentInvoices` — 12.3. Deploy the `lwc` folder.

In the code, find: label imports (`@salesforce/label/c.Portal_Pay_Now` — Chapter 9 arriving in JavaScript); the meta-XML target `lightningCommunity__Page` — **the line that makes a component appear in Experience Builder**; wire-for-read + imperative-write with `refreshApex`; and the certificate link `/sfc/servlet.shepherd/document/download/<docId>`.

12.2 The Student Portal permission set

- 1 **Setup** → **Permission Sets** → **New** — Label `Student Portal User`, API `Student_Portal_User`.
- 2 **Apex Class Access** → add `StudentPortalController` only (entry points need access; classes they call do not).
- 3 **Object Settings** → **Read only** on Courses, Cohorts, Enrolments, Invoices — and Read on every field of each. No Create/Edit anywhere: the only writer on a student's behalf is `PaymentService`, in system mode, after the gateway says yes.

12.3 Enable Digital Experiences and build the site

- 1 **Setup** → **Quick Find** `Digital Experiences` → **Settings** → tick **Enable Digital Experiences** → accept/adjust the domain → Save.
- 2 On the same page tick “**Allow using standard external profiles for self-registration, user creation, and login**” → Save. **Skip this and step 12.5 fails** with a FIELD_INTEGRITY error.
- 3 **All Sites** → **New** → template **Build Your Own (LWR)** (any works) → name `CareerLaunch Portal` → Create → wait for provisioning → open **Builder**.
- 4 Components panel (⚡, top of the left toolbar) → under **Custom Components** drag **Student Enrolments** and **Student Invoices** onto Home, side by side. Delete the template placeholders.

- 5 **Preview** shows “Your user is not linked to a student record” — that is the component **working**: an admin has no Contact. Click **Publish** (also activates the site).

12.4 Two one-time org facts

- 1 **The account owner needs a role:** Setup → Quick Find **Roles** → Set Up Roles → Expand → Add Role (e.g. **CareerLaunch Admin** under CEO) → Save. Then Setup → Users → your user → Edit → Role = CareerLaunch Admin.
- 2 **The profile must be a site member:** Experience Builder → Settings → **Administration** → **Members** → add profile **Customer Community User** → Save (re-publish if asked).

12.5 Create the demo student's login

- 1 Create Account **Oxfordable Students** (portal contacts must belong to an account), and under it Contact **Amina Bello**, email **amina.bello@oxfordable.test**.
- 2 Enrol Amina (Status Confirmed) on an open cohort — Chapter 10 raises her invoice automatically.
- 3 On the Contact: action menu (▼) → **Enable Customer User**.
- 4 Profile = **Customer Community User** · Username = globally unique, e.g. **amina.bello@careerlaunch.portal** (a username only *looks* like an email) · Email = **your real inbox** (password mails land there) → Save.
- 5 Assign the permission set: Setup → Permission Sets → Student Portal User → **Manage Assignments** → Add → Amina.
- 6 Set her password via the emailed link, or Setup → Users → Amina → **Reset Password**.

CHECKPOINT — THE FULL LOOP, AS THE STUDENT

In a **private/incognito window**, open the site URL + **/login** and sign in as Amina. **My Enrolments** shows her course (Confirmed); **My Invoices** shows one payable invoice → **Pay now** → toast “Payment received - thank you!” with a STUB reference → badge flips to Paid. As admin: invoice Paid + reference, Amount Paid = fee, zero Error Logs. Mark her Completed → the certificate flow runs → a **Download certificate** link appears in her portal. Finally switch her user language to **Français** and reload: *Mes inscriptions · Payer maintenant*.

13 Security & permissions reference

Everything access-related in one place. Rule NFR-05 from the original backlog still stands: access by permission set, never by editing a profile.

13.1 Permission set — Enrolment Console User (staff)

AREA	GRANT
Objects	Course, Cohort, Enrolment: Read/Create/Edit · Contact: Read · Invoice: Read/Create/Edit (no Delete)
Fields	Read+Edit on custom fields (Read only on formulas/roll-ups: Enrolled Count, Seats Remaining, Cohort Label, Amount In Currency)
Apex classes	<code>CohortController</code> · <code>EnrolmentController</code> · <code>CohortAvailabilityService</code> · <code>EnrolmentStatusService</code>
Tabs	Courses · Cohorts · Enrolments · Invoices (· Error Logs for team leads)
Flows	<code>Quick_Enrol_Student</code>

13.2 Permission set — Student Portal User (students)

AREA	GRANT
Objects + fields	Read only: Course, Cohort, Enrolment, Invoice (+ all their fields, read)
Apex classes	<code>StudentPortalController</code> only
Everything else	Nothing. Writes happen inside <code>PaymentService</code> (without sharing) as a system action

13.3 The portal privacy model — summary

- **Sharing sets cannot target Enrolment** (master-detail child of Cohort; Cohort has no contact field). Row privacy is enforced in Apex: contact-scoped queries + the payInvoice ownership check.
- **Fresh-org gotcha:** metadata deploys grant **no field-level security**. After any deploy to a clean org, assign both permission sets to the admin first, or `WITH SECURITY_ENFORCED` queries throw.
- Portal user prerequisites, in one line: standard-external-profiles setting ON · account owner has a role · profile is a site member · contact has an account.

14 Tests, UAT & release

Prove it works, prove it keeps working, ship it. Coverage below 75% cannot deploy to production — but coverage is the floor, not the goal: tests exist so the next person to change this code finds out immediately if they broke it.

14.1 The test suite

First add one method to the existing data factory: open `TestDataFactory` and add `createStudent(String email)` — **14.1** (the payment tests need a student with a *specific* email: `declined@...` drives the stub's failure path). Then create each test class and run it alone before moving on (`sf apex run test --tests <Name> --result-format human --wait 10`):

CLASS	CODE PACK	WHAT ITS ASSERTIONS PROVE
<code>EnrolmentTriggerTest</code>	14.2	Core: defaults · overbooking blocked (negative test) · 200-record bulk · duplicate blocked · queueable updates the Contact (<code>Test.startTest/stopTest</code> forces async)
<code>CohortServicesTest</code>	14.3	Core: controllers + both invocable services, including error paths
<code>ErrorLoggerTest</code>	14.4	Apex, Flow and custom paths all end as <code>Error_Log__c</code> rows — note <code>Test.getEventBus().deliver()</code> forces platform-event processing
<code>CurrencyServiceTest</code>	14.5	Rates, 2-dp half-up rounding, unknown-currency exception, formatting
<code>InvoicePaymentServicesTest</code>	14.6	Invoice auto-creation + idempotency · stub happy/declined/double-pay · HTTP client via <code>HttpCalloutMock</code> (asserts the <code>callout:</code> endpoint and the Idempotency-Key header, no network) · factory + override
<code>CertificateServiceTest</code>	14.7	File created + linked, flag ticked, never twice, the Flow invocable path
<code>StudentPortalControllerTest</code>	14.8	A student sees only their own records, pays their own invoice, and cannot pay someone else's — the security test that matters most

```
sf apex run test --code-coverage --result-format human --wait 10
```

TARGET

Everything green, overall coverage **75%+** (the reference build sits above 85%). Write your numbers here:
passed ____ / ____ · coverage ____ %

14.2 User Acceptance Test script

ID	TEST STEP	EXPECTED RESULT	✓
UAT-01	Create Course + linked Cohort; then set End Date before Start Date	Save works; then validation error	☐

UAT-02	One enrolment on a capacity-3 cohort	Enrolled Count 1 · Seats Remaining 2	☐
UAT-03	Enrol a 4th student on the capacity-3 cohort	Blocked with the custom capacity message	☐
UAT-04	Enrol the same student twice on one cohort	Blocked (duplicate key)	☐
UAT-05	Set an enrolment to Confirmed	Email + Task created; last seat flips Cohort to Full	☐
UAT-06	Run Quick Enrol end to end	Enrolment created; confirmation screen shows the number	☐
UAT-07	Enrol from the Enrolment Console	Toast · seat count refreshes · record created	☐
UAT-08	Confirm an enrolment on a fee-paying course; re-save it	Exactly one Sent invoice, fee amount, due +14	☐
UAT-09	Set an invoice to Paid with no reference	Validation rule blocks the save	☐
UAT-10	Pay an invoice (portal button or Apex)	Paid + STUB reference + Amount Paid updated	☐
UAT-11	Pay a declined@... student's invoice	Failure; invoice stays Sent; nothing written	☐
UAT-12	Pay the same invoice twice	Clear "already paid" error	☐
UAT-13	Execute Anonymous: <code>convertFromGBP(100, 'USD')</code>	127.00 (or 100 × your rate)	☐
UAT-14	Set an enrolment to Completed; flip status away and back	PDF filed + flag ticked, exactly once	☐
UAT-15	Force an error (divide-by-zero; a flow fault)	ERR- records with source, message, stack trace	☐
UAT-16	Log in to the portal as the demo student	Only their own enrolments and invoices	☐
UAT-17	Pay now in the portal	Success toast + reference; badge flips to Paid	☐
UAT-18	Completed course, in the portal	Download certificate opens the PDF	☐
UAT-19	Switch the portal user to French	Mes inscriptions · Payer maintenant	☐
UAT-20	Full test run	All pass · ≥75% coverage	☐

14.3 Version control & deployment to a second org

```
# retrieve anything built with clicks, then commit
sf project retrieve start --source-dir force-app
git add . && git commit -m "CareerLaunch complete build" && git push -u origin main
git tag v2.0 && git push --tags

# validate first (deploys nothing), then deploy
sf org login web --alias targetorg
sf project deploy start --source-dir force-app -o targetorg -l RunLocalTests --dry-run
sf project deploy start --source-dir force-app -o targetorg -l RunLocalTests
```

Post-deploy checklist — deployment is never finished at "the deploy succeeded"

#	ACTION
1	Assign both permission sets to the admin first — fresh deploys grant no FLS; <code>WITH SECURITY_ENFORCED</code> queries fail until you do
2	Confirm every Flow is Active (they can deploy inactive)
3	Load reference data — Courses, Cohorts, Contacts (records never deploy)
4	French: enable the language before deploying translations, or add them after
5	The Experience Cloud site is clicks, not metadata — repeat §12.3–12.5 in the target org
6	Email deliverability = All Email
7	Smoke-test UAT-08, UAT-10 and UAT-16 in the target org

GOING LIVE WITH PAYMENTS — NO CODE CHANGES

Repoint the `Payment_Gateway` Named Credential at the real provider and configure its authentication, then set `Integration_Setting__mdt → Payment_Gateway → Implementation_Class__c = PaymentGatewayHttpClient`. If the provider's JSON differs, write one new class implementing `IPaymentGateway` and point the record at that instead.

A Appendix A — Code Pack manifest

Every coded artefact, by chapter. Your instructor supplies each item as a handout; in the project repository each maps to the file shown.

ITEM	TYPE	NAME	REPOSITORY FILE
5.1	Trigger	EnrolmentTrigger	force-app/.../triggers/EnrolmentTrigger.trigger
5.2	Class	EnrolmentTriggerHandler	force-app/.../classes/EnrolmentTriggerHandler.cls
5.3	Class	StudentSummaryQueueable	force-app/.../classes/StudentSummaryQueueable.cls
5.4	Class	CohortAvailabilityService	force-app/.../classes/CohortAvailabilityService.cls
5.5	Class	EnrolmentStatusService	force-app/.../classes/EnrolmentStatusService.cls
5.6	Class	CohortController	force-app/.../classes/CohortController.cls
5.7	Class	EnrolmentController	force-app/.../classes/EnrolmentController.cls
6.1– 6.3	LWC ×3	cohortExplorer · enrolmentPanel · enrolmentConsole	force-app/.../lwc/<name>/
7.1	Class	ErrorLogger	force-app/.../classes/ErrorLogger.cls
7.2	Trigger	ErrorLogEventTrigger	force-app/.../triggers/ErrorLogEventTrigger.trigger
9.1	Class	CurrencyService	force-app/.../classes/CurrencyService.cls
10.1– 10.8	Class ×8	PaymentRequest · PaymentResult · IPaymentGateway · PaymentGatewayStub · PaymentGatewayFactory · PaymentGatewayHttpClient · InvoiceService · PaymentService	force-app/.../classes/<Name>.cls
10.9	Edit	EnrolmentTriggerHandler — the onAfterChange addition	same file as 5.2
11.1	Class	CertificateController	force-app/.../classes/CertificateController.cls
11.2	VF page	CertificatePdf	force-app/.../pages/CertificatePdf.page
11.3	Class	CertificateService	force-app/.../classes/CertificateService.cls
12.1	Class	StudentPortalController	force-app/.../classes/StudentPortalController.cls
12.2– 12.3	LWC ×2	studentEnrolments · studentInvoices	force-app/.../lwc/<name>/

14.1	Edit	TestDataFactory — add createStudent	force-app/.../classes/TestDataFactory.cls
14.2– 14.8	Test ×7	EnrolmentTriggerTest · CohortServicesTest · ErrorLoggerTest · CurrencyServiceTest · InvoicePaymentServicesTest · CertificateServiceTest · StudentPortalControllerTest	force-app/.../classes/<Name>.cls

Reference only (never handed out — students build these with clicks): objects/fields under **objects/**, tabs, permission sets, flows, custom labels (**labels/**), French translations (**translations/**), the Named Credential and custom-metadata records.

B Appendix B — Master troubleshooting table

SYMPTOM	CAUSE	FIX
Compile error on a field name	Spelling — <code>Enrollment__c</code> vs <code>Enrolment__c</code> , or an API name differing from Chapter 3/7/8	British spelling, one I ; recreate the field with the exact API name
Roll-Up Summary not offered	Child related by Lookup, not Master-Detail	Recreate the relationship as Master-Detail
Flow saved but nothing happens	Flows do nothing until activated	Open the flow → Activate
Emails never arrive	Deliverability is "System email only"	Setup → Deliverability → All Email
Too many SOQL queries: 101	A query inside a loop	Collect ids first, query once — the Chapter 5 shape
LWC missing from App Builder / Experience Builder	<code>isExposed</code> false or wrong target	Check the <code>.js-meta.xml</code> targets (<code>lightning__AppPage</code> / <code>lightningCommunity__Page</code>), redeploy, hard-refresh
LWC shows "no access"	Missing Apex class access	Add the controller to the permission set
"Log Flow Error" absent from Flow actions	ErrorLogger not compiled / invocable block missing	Re-check Code Pack 7.1
No <code>Error_Log__c</code> after the Chapter 7 checkpoint	Event trigger inactive, or event not publish-immediately	Setup → Apex Triggers; re-check §7.3
Factory error "names a class that does not exist"	Custom-metadata record typo	Value must be exactly <code>PaymentGatewayStub</code>
No invoice after confirming	Fee blank/0, or the 10.3 handler edit missing	Set a fee; re-check the one-line edit
Two invoices appeared	Service called twice or its existing-invoice query mistyped	Re-check Code Pack 10.7 / 10.9
Certificate never appears	Flow inactive — or the real cause is in Error Logs	Activate; then read the Error Logs tab
FIELD_INTEGRITY "standard external profiles" on portal user save	§12.3 step 2 skipped	Enable the setting, retry
"...account owner must have a role"	§12.4 step 1 skipped	Create + assign the role
Portal login invalid/locked	Profile not a site member, or site unpublished	§12.4 step 2; Publish

Portal cards show "not linked to a student record" for the student	User has no ContactId	Create via Enable Customer User on the Contact, not as a plain user
Queueable does not run in a test	Missing <code>Test.stopTest()</code>	Async completes at stopTest
<code>WITH SECURITY_ENFORCED</code> throws after deploy	Fresh deploys grant no FLS	Assign both permission sets to the admin
Deployment fails on coverage	Below 75%	Cover untested branches with real assertions — never empty tests

c Appendix C — Glossary

TERM	MEANING
Bulkification	Code that behaves correctly for 1 record and for 200 in one transaction: collect → one query → loop in memory
Custom Metadata Type	Admin-editable configuration records that deploy with code and are readable without SOQL
Factory	A class whose only job is deciding which implementation to hand back — here, driven by custom metadata
FLS	Field-Level Security — which fields a user can see or edit; not granted by deploys
Frozen rate	Stamping a conversion rate onto a record at creation so later edits never rewrite history
Idempotency key	A stable identifier sent with a charge so a retry can never duplicate a payment
Idempotent automation	Automation safe to re-run — e.g. the invoice service skipping enrolments that already hold a live invoice
Interface	An Apex contract with no implementation; callers depend on it instead of concrete classes
Invocable method	<code>@InvocableMethod</code> Apex that Flow (and agents) can call
Master-Detail	Tight parent-child: cascade delete, inherited sharing, roll-ups possible
Named Credential	Org record owning an endpoint + auth; Apex says <code>callout:Name</code> , never a URL or secret
Platform event (publish immediately)	Event-bus message that survives its transaction's rollback — the backbone of Chapter 7
Queueable (+ AllowsCallouts)	Async Apex with fresh limits; the only place <code>getContent()</code> may run
<code>renderAs="pdf"</code>	Visualforce attribute rendering the page through the platform PDF engine
Sharing set	Experience Cloud record-access grant for community users — cannot target master-detail children, hence the Chapter 12 design
Stub	A deterministic fake implementation used where the real system is unreachable or unwanted
UAT	User Acceptance Testing — the business confirming the solution does what was asked